Lecture 8: Strong **NP**-completeness, **NP** and **coNP**

Dr. George Mertzios

george.mertzios@durham.ac.uk

Room 2066, MCS Building
Tel: 42429

KNAPSACK

KNAPSACK
Instance: A set of items $I = \{i_1, \ldots i_n\}$ where each item i_j has value v_j and weight w_j (values and weights are positive integers); and a target value t and a weight upper limit L.
Question: Is there a subset T of I with total value at least t and total weight at most L.

Theorem

KNAPSACK *is* **NP**-*complete.*

- Step 1. The problem is in **NP**: a subset T is the certificate.
- Step 2. To show completeness, reduce SUBSET SUM to KNAPSACK. We will use the method of "restriction".

Reduction

SUBSET SUM
Instance: A set of positive integers $S = \{n_1, n_2, \ldots, n_k\}$, and a target integer t'.
Question: Is there a subset $T \subseteq S$, such that $\sum_{i \in T} n_i = t'$?

- Consider instances of KNAPSACK such that $v_j = w_j (= n_j)$ for all j, and $t = L (= t')$. Then we get exactly SUBSET SUM.
- Indeed, we have both $\sum_{j \in T} v_j \geq t$ and $\sum_{j \in T} w_j \leq L$ if and only if $\sum_{j \in T} n_j = t'$.

A polynomial algorithm?

KNAPSACK can be solved in $O(nL)$ time using dynamic programming

- Create a $(L+1) \times (n+1)$ matrix M
- Set $M(w,0) = M(0,i)$ to 0 for all $w \in \{1,2,\ldots,L\}$ and all $i \in \{1,2,\ldots,n\}$
- For $i = 0,1,\ldots,n-1$, set entry $M(w,i+1)$ as follows

$$M(w,i+1) = \max\{M(w,i),\ M(w-w_{i+1},i) + v_{i+1}\}$$

  ($M(w,i)$ is the largest total value obtainable by selecting from the first i items with weight limit w)
- Answer yes if any entry $\geq t$, otherwise no

Example construction

$M(w, i+1) = \max\{M(w, i),\ M(w - w_{i+1}, i) + v_{i+1}\}$

Let $n = 4$; values $1, 3, 4, 2$; weights $1, 1, 3, 2$; target $t = 7$ and limit $L = 5$.

weight limit w	max total value from first i items				
	$i = 0$	$i = 1$	$i = 2$	$i = 3$	$i = 4$
0					
1					
2					
3					
4					
5					X

Example construction

$M(w, i+1) = \max\{M(w, i),\ M(w - w_{i+1}, i) + v_{i+1}\}$

Let $n = 4$; values $1, 3, 4, 2$; weights $1, 1, 3, 2$; target $t = 7$ and limit $L = 5$.

weight limit w	max total value from first i items				
	$i = 0$	$i = 1$	$i = 2$	$i = 3$	$i = 4$
0	0	0	0	0	0
1	0				
2	0				
3	0				
4	0				
5	0				

Example construction

$M(w, i+1) = \max\{M(w, i),\ M(w - w_{i+1}, i) + v_{i+1}\}$

Let $n = 4$; values $1, 3, 4, 2$; weights $1, 1, 3, 2$; target $t = 7$ and limit $L = 5$.

weight limit w	max total value from first i items				
	$i = 0$	$i = 1$	$i = 2$	$i = 3$	$i = 4$
0	0	0	0	0	0
1	0	1	3	3	3
2	0	1	4	4	X
3	0				
4	0				
5	0				

Example construction

$M(w, i+1) = \max\{M(w, i),\ M(w - w_{i+1}, i) + v_{i+1}\}$

Let $n = 4$; values $1, 3, 4, 2$; weights $1, 1, 3, 2$; target $t = 7$ and limit $L = 5$.

weight limit w	max total value from first i items				
	$i = 0$	$i = 1$	$i = 2$	$i = 3$	$i = 4$
0	0	0	0	0	0
1	0	1	3	3	3
2	0	1	4	4	X
3	0				
4	0				
5	0				

Example construction

$M(w, i+1) = \max\{M(w, i),\ M(w - w_{i+1}, i) + v_{i+1}\}$

Let $n = 4$; values $1, 3, 4, 2$; weights $1, 1, 3, 2$; target $t = 7$ and limit $L = 5$.

weight limit w	max total value from first i items				
	$i = 0$	$i = 1$	$i = 2$	$i = 3$	$i = 4$
0	0	0	0	0	0
1	0	1	3	3	3
2	0	1	4	4	X
3	0				
4	0				
5	0				

Example construction

$M(w, i+1) = \max\{M(w, i),\ M(w - w_{i+1}, i) + v_{i+1}\}$

Let $n = 4$; values $1, 3, 4, 2$; weights $1, 1, 3, 2$; target $t = 7$ and limit $L = 5$.

weight limit w	max total value from first i items				
	$i = 0$	$i = 1$	$i = 2$	$i = 3$	$i = 4$
0	0	0	0	0	0
1	0	1	3	3	3
2	0	1	4	4	4
3	0				
4	0				
5	0				

Example construction

$M(w, i+1) = \max\{M(w, i),\ M(w - w_{i+1}, i) + v_{i+1}\}$

Let $n = 4$; values $1, 3, 4, 2$; weights $1, 1, 3, 2$; target $t = 7$ and limit $L = 5$.

weight limit w	max total value from first i items				
	$i = 0$	$i = 1$	$i = 2$	$i = 3$	$i = 4$
0	0	0	0	0	0
1	0	1	3	3	3
2	0	1	4	4	4
3	0	1	4	4	5
4	0	1	4	X	
5	0				

Example construction

$M(w, i+1) = \max\{M(w, i),\ M(w - w_{i+1}, i) + v_{i+1}\}$

Let $n = 4$; values $1, 3, 4, 2$; weights $1, 1, 3, 2$; target $t = 7$ and limit $L = 5$.

weight limit w	max total value from first i items				
	$i = 0$	$i = 1$	$i = 2$	$i = 3$	$i = 4$
0	0	0	0	0	0
1	0	1	3	3	3
2	0	1	4	4	4
3	0	1	4	4	5
4	0	1	4	X	
5	0				

Example construction

$M(w, i+1) = \max\{M(w, i),\ M(w - w_{i+1}, i) + v_{i+1}\}$

Let $n = 4$; values $1, 3, 4, 2$; weights $1, 1, 3, 2$; target $t = 7$ and limit $L = 5$.

weight limit w	max total value from first i items				
	$i = 0$	$i = 1$	$i = 2$	$i = 3$	$i = 4$
0	0	0	0	0	0
1	0	1	3	3	3
2	0	1	4	4	4
3	0	1	4	4	5
4	0	1	4	X	
5	0				

Example construction

$M(w, i+1) = \max\{M(w, i),\ M(w - w_{i+1}, i) + v_{i+1}\}$

Let $n = 4$; values $1, 3, 4, 2$; weights $1, 1, 3, 2$; target $t = 7$ and limit $L = 5$.

weight limit w	max total value from first i items				
	$i = 0$	$i = 1$	$i = 2$	$i = 3$	$i = 4$
0	0	0	0	0	0
1	0	1	3	3	3
2	0	1	4	4	4
3	0	1	4	4	5
4	0	1	4	7	
5	0				

Example construction

$M(w, i+1) = \max\{M(w, i),\ M(w - w_{i+1}, i) + v_{i+1}\}$

Let $n = 4$; values $1, 3, 4, 2$; weights $1, 1, 3, 2$; target $t = 7$ and limit $L = 5$.

weight limit w	max total value from first i items				
	$i = 0$	$i = 1$	$i = 2$	$i = 3$	$i = 4$
0	0	0	0	0	0
1	0	1	3	3	3
2	0	1	4	4	4
3	0	1	4	4	5
4	0	1	4	7	7
5	0	1	4	8	8

Pseudo-polynomial time

This algorithm is not sufficient to show that KNAPSACK is in **P** ! Why?

Pseudo-polynomial time

This algorithm is not sufficient to show that KNAPSACK is in **P** ! Why?

The length of the input to KNAPSACK is in $O(n \log L) \Rightarrow nL$ is not bounded by a polynomial function of the input length.
(This is a typical source of confusion in amateur "proofs" that **P** $=$ **NP**: numbers must be encoded in binary, not in unary!)

Pseudo-polynomial time

This algorithm is not sufficient to show that KNAPSACK is in **P** ! Why?

The length of the input to KNAPSACK is in $O(n \log L) \Rightarrow nL$ is not bounded by a polynomial function of the input length.
(This is a typical source of confusion in amateur "proofs" that **P** = **NP**: numbers must be encoded in binary, not in unary!)

However, if we restrict KNAPSACK to instances such that $L \leq p(n)$ for some polynomial p, then we obtain a problem in **P**.

- An algorithm which is polynomial in the size of the numbers in the input is called pseudo-polynomial
- NP-complete problems which remain NP-complete when all numbers are bounded by some polynomial in the length of the input are called strongly NP-complete
  (otherwise they are called weakly NP-complete)

Strongly **NP**-complete problems

- Any **NP**-complete problem without numerical data is strongly **NP**-complete: e.g., SATISFIABILITY, HAMILTONIAN CYCLE, 3-COLOURABILITY, VERTEX COVER, INDEPENDENT SET, CLIQUE...
- TSP (D) is strongly **NP**-complete, since it is **NP**-complete even if all distances are 1 and 2 (see reduction from HAMILTONIAN CYCLE).
- SUBSET SUM is not strongly **NP**-complete, it is a subproblem of KNAPSACK (i.e. "easier" than KNAPSACK).

Examples

PARTITION
Instance: A multi-set S of positive integers (i.e. some integers may appear multiple times in S).
Question: Is there a partition of S into two subsets S_1, S_2 such that $\sum_{s \in S_1} s = \sum_{s \in S_2} s$?

Examples

PARTITION
Instance: A multi-set S of positive integers (i.e. some integers may appear multiple times in S).
Question: Is there a partition of S into two subsets S_1, S_2 such that $\sum_{s \in S_1} s = \sum_{s \in S_2} s$?

- This problem is (weakly) **NP**-complete
- A dynamic programming algorithm with running time $O(|S| \cdot \sum_{s \in S} s)$
- The length of the input is $O(|S| + \sum_{s \in S} \log s)$

Examples

PARTITION
Instance: A multi-set S of positive integers (i.e. some integers may appear multiple times in S).
Question: Is there a partition of S into two subsets S_1, S_2 such that $\sum_{s \in S_1} s = \sum_{s \in S_2} s$?

- This problem is (weakly) **NP**-complete
- A dynamic programming algorithm with running time $O(|S| \cdot \sum_{s \in S} s)$
- The length of the input is $O(|S| + \sum_{s \in S} \log s)$
- This running time is pseudo-polynomial (but not polynomial!)
  - if we encode the input numbers in unary (large encoding) $\Rightarrow$ running time is small (compared with the input size)
  - if we encode the input numbers in binary (succinct encoding) $\Rightarrow$ running time is large (compared with the input size)

Examples

3-PARTITION
Instance: A multi-set S of with $n = 3m$ positive integers.
Question: Is there a partition of S into m subsets $S_1, S_2, \ldots, S_m$, such that $\sum_{s \in S_i} s = \sum_{s \in S_j} s$ for every $i, j \leq m$?

Examples

3-PARTITION
Instance: A multi-set S of with $n = 3m$ positive integers.
Question: Is there a partition of S into m subsets $S_1, S_2, \ldots, S_m$, such that $\sum_{s \in S_i} s = \sum_{s \in S_j} s$ for every $i, j \leq m$?

- Similar problem to PARTITION, but still very different !
- It is strongly **NP**-complete
  - the order of the time complexity does not change much if we represent the numbers in unary (large encoding) or in binary (succinct encoding)

Examples

3-PARTITION
Instance: A multi-set S of with $n = 3m$ positive integers.
Question: Is there a partition of S into m subsets $S_1, S_2, \ldots, S_m$, such that $\sum_{s \in S_i} s = \sum_{s \in S_j} s$ for every $i, j \leq m$?

- Similar problem to PARTITION, but still very different !
- It is strongly **NP**-complete
  - the order of the time complexity does not change much if we represent the numbers in unary (large encoding) or in binary (succinct encoding)
- Let $B = \frac{1}{m} \cdot \sum_{s \in S} s$
- It remains (strongly) **NP**-complete even if $\frac{B}{4} < s < \frac{B}{2}, \forall s \in S$
  - in this case every set S_i has exactly 3 elements
- There exists no (pseudo-) polynomial algorithm (unless **P**$=$ **NP**)

NP and **coNP**

For a language (i.e. problem) $\mathcal{L}$ over alphabet Σ, let $\overline{\mathcal{L}}$ denote the *complement* $\Sigma^* \setminus \mathcal{L}$ of $\mathcal{L}$ (that is, $x \in \overline{\mathcal{L}} \Leftrightarrow x \notin \mathcal{L}$).

Definition

The class of languages $\mathcal{L}$ such that $\overline{\mathcal{L}}$ has a polynomial-time verifier is called **coNP**.

In other words, a problem belongs to **coNP** if the **no**-instances have succinct certificates

NP and **coNP**

For a language (i.e. problem) $\mathcal{L}$ over alphabet Σ, let $\overline{\mathcal{L}}$ denote the *complement* $\Sigma^* \setminus \mathcal{L}$ of $\mathcal{L}$ (that is, $x \in \overline{\mathcal{L}} \Leftrightarrow x \notin \mathcal{L}$).

Definition

The class of languages $\mathcal{L}$ such that $\overline{\mathcal{L}}$ has a polynomial-time verifier is called **coNP**.

In other words, a problem belongs to **coNP** if the **no**-instances have succinct certificates

Example:

NO HAMILTONIAN CYCLE
Instance: A graph G
Question: Is it true that G has no Hamiltonian cycle?

- A Hamiltonian Cycle (if it exists!) can be verified efficiently

$\Rightarrow$ a Hamiltonian Cycle is a certificate for no-instances

Example: TAUTOLOGY

TAUTOLOGY
Instance: A DNF-formula f
Question: Is it true that f is a tautology, that is, is f satisfied by all assignments?

- A truth assignment that makes f FALSE can be verified efficiently

$\Rightarrow$ such a truth assignment is a certificate for no-instances

Example: TAUTOLOGY

TAUTOLOGY
Instance: A DNF-formula f
Question: Is it true that f is a tautology, that is, is f satisfied by all assignments?

- A truth assignment that makes f FALSE can be verified efficiently

$\Rightarrow$ such a truth assignment is a certificate for no-instances

- Is TAUTOLOGY in **NP**? Is it easy to verify that all assignments satisfy f?

Example: TAUTOLOGY

TAUTOLOGY
Instance: A DNF-formula f
Question: Is it true that f is a tautology, that is, is f satisfied by all assignments?

An equivalent formulation of this problem:

TAUTOLOGY (VARIANT 2)
Instance: A DNF-formula f
Question: Is it true that there does not exist a truth assignment that un-satisfies f?

Example: TAUTOLOGY

TAUTOLOGY
Instance: A DNF-formula f
Question: Is it true that f is a tautology, that is, is f satisfied by all assignments?

An equivalent formulation of this problem:

TAUTOLOGY (VARIANT 2)
Instance: A DNF-formula f
Question: Is it true that there does not exist a truth assignment that un-satisfies f?

COMPLEMENT OF TAUTOLOGY
Instance: A DNF-formula f
Question: Does there exist a truth assignment that un-satisfies f?

Example: TAUTOLOGY

TAUTOLOGY
Instance: A DNF-formula f
Question: Is it true that f is a tautology, that is, is f satisfied by all assignments?

An equivalent formulation of this problem:

TAUTOLOGY (VARIANT 2)
Instance: A DNF-formula f
Question: Is it true that there does not exist a truth assignment that un-satisfies f?

COMPLEMENT OF TAUTOLOGY
Instance: A DNF-formula f
Question: Does there exist a truth assignment that un-satisfies f?

Question: Have you seen the last problem with a different name?

Example: TAUTOLOGY

TAUTOLOGY
Instance: A DNF-formula f
Question: Is it true that f is a tautology, that is, is f satisfied by all assignments?

An equivalent formulation of this problem:

TAUTOLOGY (VARIANT 2)
Instance: A DNF-formula f
Question: Is it true that there does not exist a truth assignment that un-satisfies f?

COMPLEMENT OF TAUTOLOGY
Instance: A DNF-formula f
Question: Does there exist a truth assignment that un-satisfies f?

Question: Have you seen the last problem with a different name?
Answer: SATISFIABILITY !!!

coNP-completeness

Definition

A language $\mathcal{L} \in$ **coNP** is said to be **coNP**-complete if, for any language $\mathcal{L}' \in$ **coNP**, we have $\mathcal{L}' \leq \mathcal{L}$.

coNP-completeness

Definition

A language $\mathcal{L} \in$ **coNP** is said to be **coNP**-complete if, for any language $\mathcal{L}' \in$ **coNP**, we have $\mathcal{L}' \leq \mathcal{L}$.

Theorem

A language $\mathcal{L}$ is **NP***-complete if and only if $\overline{\mathcal{L}}$ is* **coNP***-complete.*

Proof: We will show one direction, the other is similar. Let $\mathcal{L}$ be **NP**-complete and choose any $\mathcal{L}_0 \in$ **coNP**. Then $\overline{\mathcal{L}_0}$ is in **NP**. So there is a polynomial-time reduction R from $\overline{\mathcal{L}_0}$ to $\mathcal{L}$ (i.e. $\overline{\mathcal{L}_0} \leq \mathcal{L}$). Then R is also a reduction from $\mathcal{L}_0$ to $\overline{\mathcal{L}}$:

$$x \in \mathcal{L}_0 \Leftrightarrow x \notin \overline{\mathcal{L}_0} \Leftrightarrow R(x) \notin \mathcal{L} \Leftrightarrow R(x) \in \overline{\mathcal{L}}. \qquad \square$$

coNP-completeness

Definition

A language $\mathcal{L} \in$ **coNP** is said to be **coNP**-complete if, for any language $\mathcal{L}' \in$ **coNP**, we have $\mathcal{L}' \leq \mathcal{L}$.

Theorem

A language $\mathcal{L}$ is **NP***-complete if and only if $\overline{\mathcal{L}}$ is* **coNP***-complete.*

Proof: We will show one direction, the other is similar. Let $\mathcal{L}$ be **NP**-complete and choose any $\mathcal{L}_0 \in$ **coNP**. Then $\overline{\mathcal{L}_0}$ is in **NP**. So there is a polynomial-time reduction R from $\overline{\mathcal{L}_0}$ to $\mathcal{L}$ (i.e. $\overline{\mathcal{L}_0} \leq \mathcal{L}$). Then R is also a reduction from $\mathcal{L}_0$ to $\overline{\mathcal{L}}$:

$$x \in \mathcal{L}_0 \Leftrightarrow x \notin \overline{\mathcal{L}_0} \Leftrightarrow R(x) \notin \mathcal{L} \Leftrightarrow R(x) \in \overline{\mathcal{L}}. \qquad \square$$

Corollary

The problems NO HAM CYCLE *and* TAUTOLOGY *are* **coNP***-complete.*

NP vs. coNP

Theorem

If some **coNP***-complete problem belongs to* **NP***, then* **NP** $=$ **coNP***.*

Proof: If $\mathcal{L} \in$ **NP** is **coNP**-complete then every $\mathcal{L}_0 \in$ **coNP** is polynomial-time reducible to $\mathcal{L}$ (i.e. $\mathcal{L}_0 \leq \mathcal{L}$). Thus, since $\mathcal{L} \in$ **NP**, it follows that $\mathcal{L}_0 \in$ **NP** and thus **coNP** $\subseteq$ **NP**.

By the theorem from the previous slide, $\overline{\mathcal{L}} \in$ **coNP** is **NP**-complete. Since every problem from **NP** is reducible to $\overline{\mathcal{L}}$, it follows that **NP** $\subseteq$ **coNP**. $\square$

NP vs. coNP

Theorem

If some **coNP***-complete problem belongs to* **NP***, then* **NP** $=$ **coNP***.*

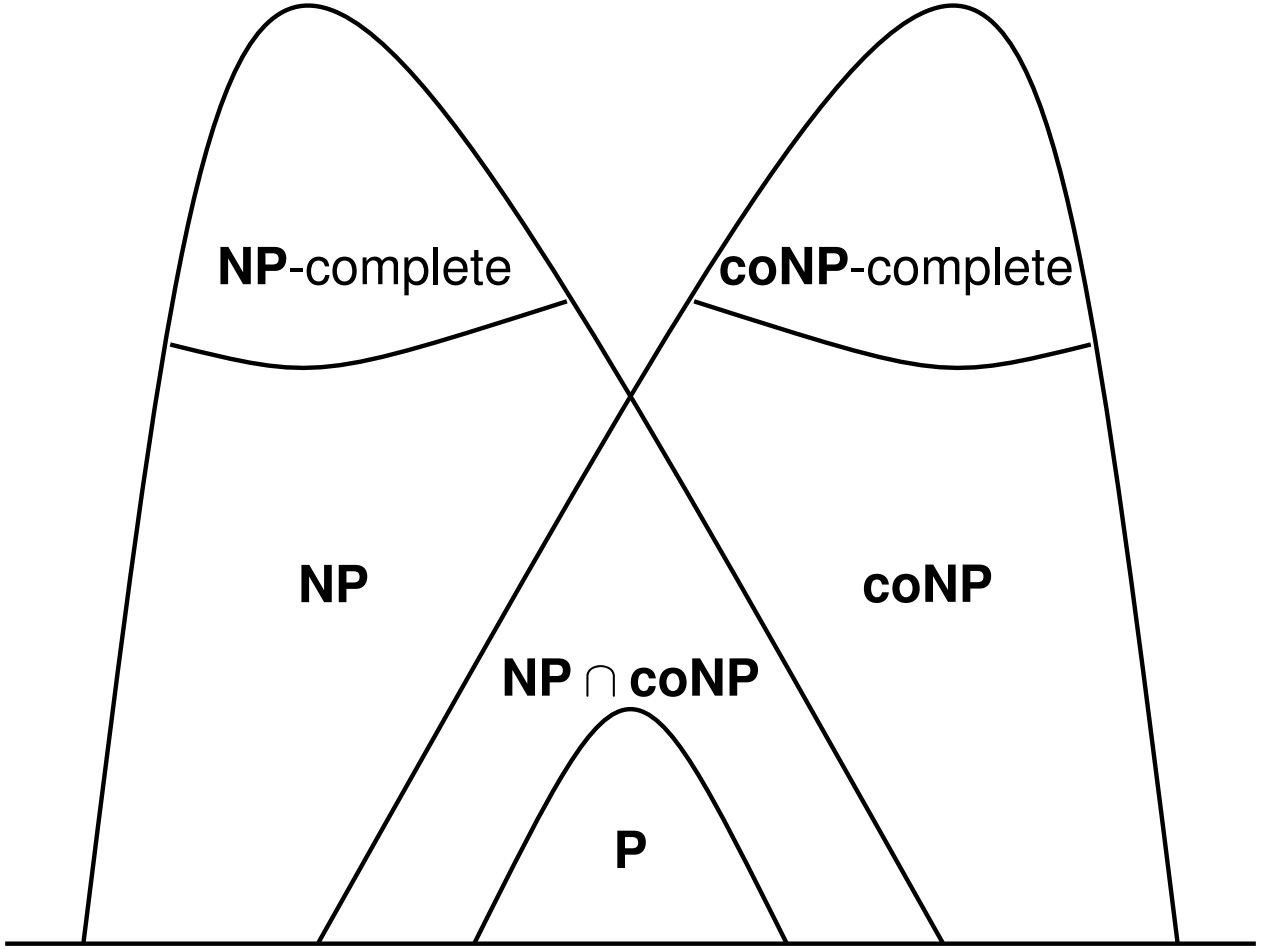
Proof: If $\mathcal{L} \in$ **NP** is **coNP**-complete then every $\mathcal{L}_0 \in$ **coNP** is polynomial-time reducible to $\mathcal{L}$ (i.e. $\mathcal{L}_0 \leq \mathcal{L}$). Thus, since $\mathcal{L} \in$ **NP**, it follows that $\mathcal{L}_0 \in$ **NP** and thus **coNP** $\subseteq$ **NP**.

By the theorem from the previous slide, $\overline{\mathcal{L}} \in$ **coNP** is **NP**-complete. Since every problem from **NP** is reducible to $\overline{\mathcal{L}}$, it follows that **NP** $\subseteq$ **coNP**. $\square$

Corollary

If **NP** $\neq$ **coNP** *then* TAUTOLOGY *and* NO HAM CYCLE *do not belong to* **NP***.*

Remarks

- It holds that $\mathbf{P} = \mathbf{coP}$. (Why?)
- Clearly, if $\mathbf{P} = \mathbf{NP}$ then $\mathbf{NP} = \mathbf{coNP}$. However, it is still possible that $\mathbf{NP} = \mathbf{coNP}$ and $\mathbf{P} \neq \mathbf{NP}$.
- It is widely believed that $\mathbf{P} \neq \mathbf{NP}$ and $\mathbf{NP} \neq \mathbf{coNP}$.
- We also have $\mathbf{P} \subseteq \mathbf{NP} \cap \mathbf{coNP}$. It is not known whether this inclusion is strict, but most specialists tend to think it is.
- In many textbooks on complexity, the problem PRIMES (decide whether a given integer is prime) is given as an example of a problem in $\mathbf{NP} \cap \mathbf{coNP}$, and it was open for decades whether this problem is in $\mathbf{P}$.
- In 2002, M. Agrawal and two of his undergraduate students proved that PRIMES is in $\mathbf{P}$. Their algorithm is surprisingly simple, see `tinyurl.com/695dj7`.

Around **NP**